Artificial Intelligence 2
Exercise Sheet 2

Johannes P. Wallner

Individual Exercises (5 P)

- Visit `https://checkr.tugraz.at/` (a TU Graz software, login using your TU Graz account).
- At `https://checkr.tugraz.at/` enter the key 8ydg5H (same as for first exercise sheet).
- Provide your answers to the posed questions - several tries are possible (20 mins waiting time).
- If all questions have been answered correctly, this corresponds to 5 points (each correctly answered question corresponds to $\frac{1}{3}$ point).
- No explicit "Abgabe" is needed for this part (just answer the questions in KnowledgeCheckR).

Submission and Files

In the exercises below you will be asked to submit files in TeachCenter. Submit the files separately, not in archived ("zipped") form. These are the files to submit:

- Part A: exercise2a.pdf, circuit.dimacs, circuit-observation1.dimacs, diagnosis1.wdimacs, and diagnosis2.wdimacs
- Part B: exercise2b.pdf, instance.asp, placementA.asp, and placementB.asp

In general, exercise2a.pdf and exercise2b.pdf must contain your answers to the exercises, and explanations. The other files contain code (DIMACS and answer set programming). **Explain in your own words** your solutions in exercise2a.pdf and exercise2b.pdf.

In the following exercises, please follow *exactly* the specification. We will use partially automated test cases for evaluation. If these test cases fail, your scores will likely be reduced.

Part A: Model-based Diagnosis

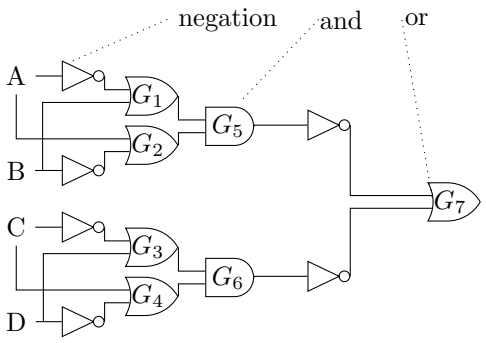


Figure 1: A circuit

In this exercise we are going to represent a circuit in Boolean logic, and apply model-based diagnosis. This task is divided into some subtasks. The circuit we are going to consider is shown in Figure 1. This circuit has four Boolean inputs (A, B, C, and D), and an output gate G_7. The five gates, G_1, G_2, G_3, G_4, and G_7 are logical disjunctions and the gates G_5 and G_6 are logical conjunctions. Note that some inputs to G_1, G_2, G_3, G_4, and G_7 are negated (logical negation).

Representing the Circuit in Boolean Logic (2 P)

First, represent the circuit as a system description SD, as shown in the lecture. That is, represent gates via logical equivalences, and use abnormality variables for each gate. Use *exactly* the 18 propositional variables $P = \{A, B, C, D, G_1, G_2, G_3, G_4, G_5, G_6, G_7, Ab_{G_1}, Ab_{G_2}, Ab_{G_3}, Ab_{G_4}, Ab_{G_5}, Ab_{G_6}, Ab_{G_7}\}$. The components are the seven gates ($Comps = \{G_1, G_2, G_3, G_4, G_5, G_6, G_7\}$). Write down the system description in conjunctive normal form. Do not use further auxiliary variables (like Tseitin variables).

1. First, write this formula in exercise2a.pdf and
2. write down the formula in DIMACS format in file circuit.dimacs.

Use the following identifiers for the variables in the DIMACS files.

A	1	G_1	5	Ab_{G_1}	12
B	2	G_2	6	Ab_{G_2}	13
C	3	G_3	7	Ab_{G_3}	14
D	4	G_4	8	Ab_{G_4}	15
		G_5	9	Ab_{G_5}	16
		G_6	10	Ab_{G_6}	17
		G_7	11	Ab_{G_7}	18

Minimum-Cardinality Diagnoses (5 P)

Consider the following three observations:

1. $Obs_1 = G_7 \wedge \neg Ab_{G_1} \wedge \neg Ab_{G_2} \wedge \neg Ab_{G_3} \wedge \neg Ab_{G_4} \wedge \neg Ab_{G_5} \wedge \neg Ab_{G_6} \wedge \neg Ab_{G_7}$ (circuit evaluates to true and all gates are normal),
2. $Obs_2 = A \wedge B \wedge \neg C \wedge \neg D \wedge G_7$ (some observations on input and output),
3. $Obs_3 = A \wedge \neg B \wedge C \wedge \neg D \wedge \neg G_7 \wedge \neg Ab_{G_7}$ (observations on input, output, and gate G_7 is normal).

For the first observation, extend your encoding from the previous task with Obs_1, i.e., write down $SD \wedge Obs_1$ in a file circuit-observation1.dimacs.

Encode the model-based diagnosis problem $(SD, Comps)$ with Obs_2 (second observation) in MaxSAT, as shown in the lecture (with unit cost to the soft clauses). Write this MaxSAT formula in diagnosis1.wdimacs.

Furthermore, encode the model-based diagnosis problem $(SD, Comps)$ with Obs_3 (third observation) in MaxSAT, again as shown in the lecture with unit costs for the soft clauses). Write this in diagnosis2.wdimacs. Answer the following questions in exercise2a.pdf.

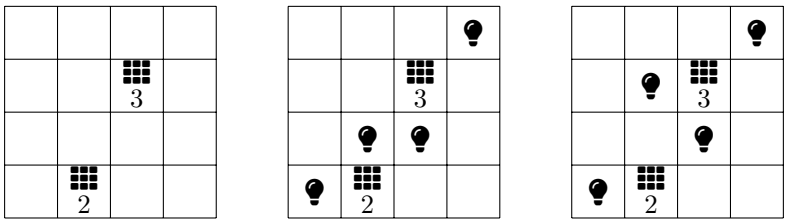
- How many models are there for SD? (without any observations)
- How many models are there for $SD \wedge Obs_1$?
- Write down all minimum-cardinality diagnoses for $(SD, Comps)$ with Obs_2 (using your MaxSAT formalization).
- Give one minimum-cardinality diagnosis for $(SD, Comps)$ with Obs_3.

Part B: Answer Set Programming (8 P)

In this exercise we are going to model a variant of a (logical) game called "Light Up" in answer set programming (ASP).

In this game, we are given an $n \times n$ grid, as shown in Figure 2. We number the rows from top to bottom with 1 to n, and, likewise, the columns from left to right with 1 to n. Additionally, the input consists of a placement of some walls and all walls have an associated positive integer (called capacity) from 1 to 8. For instance, in Figure 2(a), we have a 4×4 grid, with two walls at $(4, 2)$ (fourth row and second column) and at $(2, 3)$. The former one has a capacity of 2 the latter 3.

In this game we have to place light bulbs. A light bulb can be placed into a cell without a wall. A light bulb placed into a cell lights up its own cell and all cells connected horizontally and vertically up to an obstacle. An obstacle can be a wall or the end of the grid. For instance, in the placement in Figure 2(c), the light bulb in $(2, 2)$ lights up the cell $(2, 1)$ (to the left of the bulb), the cell $(1, 2)$ (to the top of the bulb), the cell $(3, 2)$ (below the bulb), and the cell with



(a) (b) (c)

Figure 2: A specific input to the Light Up game (left), a solution for Task A (disregarding capacities), and a solution for Task B (respecting capacities)..

the bulb itself $(2, 2)$. This light bulb does not light up cells with walls, like $(2, 3)$ and $(4, 2)$, and, moreover, also does not light up cell $(2, 4)$, since one of the walls blocks the light.

We consider two (connected) tasks. The first task (Task A) is to place light bulbs such that each non-wall cell is lit up and, additionally, a light bulb can only be placed into cells adjacent to cells with a wall. Capacity is not used for Task A. Two cells are adjacent if they are directly connected horizontally, vertically, or diagonally (that is, a light bulb can be placed "one step" apart from a wall). In Figure 2(b) there is a placement satisfying these constraints.

In the second task (Task B) the same conditions apply as for Task A, however, additionally, for each wall with a capacity k *at most* k-many light bulbs can be placed in adjacent cells. Note that this "at most" statement also applies in case two walls are close by. For instance in Figure 2(c) there is one such placement. Placement in Figure 2(b) does not satisfy these constraints, since the lower wall has capacity 2, but there are three light bulbs in adjacent cells.

Your task is to model placement of light bulbs in ASP. We model this in ASP in the following way. Each task is represented in one file (placementA.asp and placementB.asp), and the given grid in another file.

You are required to use the following names of predicates.

- row/1 and col/1 represent the available rows and columns, e.g., for dimension 2×2 we have row(1), row(2), col(1), and col(2)
- wall/3 represents where a wall is placed, e.g., wall(2,3,5) means a wall at row 2 and column 3 with capacity 5
- placeLightBulb/2 represents a placement of a bulb, e.g., placeLightBulb(1,2) represents placing a light bulb at the cell in row 1 and column 2

The tasks are then as follows.

- Write an ASP, in placementA.asp, such that each answer set corresponds to one placement of light bulbs (according to the rules above) for Task A.

- Write an ASP, in placementB.asp, such that each answer set corresponds to one placement of light bulbs (according to the rules above) for Task B.
- For testing your solution, copy the instance given below to a file (test-input.asp) and run "clingo test-input.asp placementA.asp". This should give an answer set corresponding to the placement shown in the figure.
- In exercise2b.pdf describe your solution (as specified in the beginning of this sheet).

Note that your ASP encodings (placementA.asp and placementB.asp) are to be used in the following way: "clingo instance.asp placementA.asp" and "clingo instance.asp placementB.asp" for the two different tasks, with "instance.asp" encoding the instance of a specific grid. For instance test-input.asp (see below) represents the instance for the grid in Figure 2. In other words, your placementA.asp and placementB.asp contain the ASP for solving the Light Up problem for any grid instance, which is then specified in another file.

```
% Defining the rows and columns.
row(1..4).
col(1..4).
% walls and their capacity
wall(2,3,3).
wall(4,2,2).
```